

Department of Computer Science
University of Pretoria

Programming Languages
COS 333

Practical 7:
Object-Oriented Programming

13 May, 2026

1 Objectives

This practical aims to achieve the following general learning objectives:

- To gain and consolidate some experience writing object-oriented programs in Ruby;
- To consolidate a variety of basic concepts related to object-oriented programming languages, as presented in the prescribed textbook for this course.

2 Plagiarism Policy

Plagiarism is a serious form of academic misconduct. It involves both appropriating someone else's work and passing it off as one's own work afterwards. Thus, you commit plagiarism when you present someone else's written or creative work (words, images, ideas, opinions, discoveries, artwork, music, recordings, computer-generated work, etc.) as your own. Note that using material produced in whole or part by an AI-based tool (such as ChatGPT) also constitutes plagiarism. Only hand in your own original work. Indicate precisely and accurately when you have used information provided by someone else. Referencing must be done in accordance with a recognised system. Indicate whether you have downloaded information from the Internet. For more details, visit the library's website: <http://www.library.up.ac.za/plagiarism/>.

3 Submission Instructions

Upload your practical-related source code file to the appropriate assignment upload slots on the ClickUP course page. Your Ruby implementation must include your entire Ruby program in a single source file named `u99999999.rb`, where 99999999 is your student number. Multiple uploads are allowed, but only the last one will be marked. The submission deadline is **Friday, 22 May 2026, at 12:00** (note that marking will take place in the weeks of 18 May 2026 and 25 May 2026, so you are welcome to have the practical marked before the deadline).

4 Background Information

For this practical, you will be writing a program in Ruby 3.4. The course website contains documentation related to Ruby [1]. For this implementation, you will be writing all classes in a single source file.

You may use any features or libraries included as part of the standard installations of Ruby. **However, note that you may not use any third-party extensions that require additional downloads.** Use of any third-party extensions will result in a zero mark for the task in question.

5 Practical Task

This practical requires you to build a simple system in Ruby, which represents characters in a computer game. This implementation must use object-oriented programming concepts. The following details must be included in your implementation:

A **GameCharacter** class, which represents a generic game character. This class is not meant to be instantiated. In a programming language like C++ or Java, this class would be abstract to prevent its instantiation. However, note that Ruby does not directly support abstract classes. The class should have the following characteristics:

- One instance variable: **health**, which is an integer value representing the number of health points of the game character.
- A constructor that receives one parameter, which is used to set the value of **health** for the game character.
- An accessor method that returns the value of the **health** instance variable. In Ruby, provide an attribute reader instead of a separate accessor method.
- An instance method, called **takeHit** that takes a single integer parameter and returns a Boolean value. The Boolean value indicates whether the **GameCharacter** is still alive (if the value is true) or dead (if the value is false). This method should not be implemented in the **GameCharacter** class.
- An instance method, called **attack** that takes no parameters and returns an integer value. This method should return a value of 1.

A derived classes of **GameCharacter**, called **Wizard**. Instances of this class can be created. The class should have the following characteristics:

- In addition to all the instance variable inherited from the **GameCharacter** class, each **Wizard** has one additional instance variable: **spellDamage**, which is an integer value representing an attack damage bonus.
- A constructor that receives two parameters, which are used to set the **health** and **spellDamage** instance variables. The constructor should call the parent class constructor, in order to set the value of the inherited instance variable.
- The class overrides the **takeHit** method inherited from the parent class. This method simply returns false (because a **Wizard** is instantly killed by any attack).
- The class overrides the **attack** method inherited from the parent class. This method returns the **spellDamage** instance variable value. The parent class method is not called.

Another derived class of **GameCharacter**, called **Warrior**. Instances of this class can be created. The class should have the following characteristics:

- In addition to the instance variable inherited from the **GameCharacter** class, each **Warrior** has one additional instance variable: **shield**, which is an integer value representing the amount of damage the warrior's shield absorbs.
- A constructor that receives two parameters, which are used to set the **health** and **shield** instance variables. The constructor should call the parent class constructor, in order to set values of the inherited instance variable.
- The class overrides the **takeHit** method inherited from the parent class. This method subtracts the **shield** instance variable from the parameter value, giving the amount of damage not absorbed by the shield. The **health** member variable is then decremented by the damage not absorbed by the shield. Finally, if the value of **health** is negative, the method returns false (because all the remaining health has been depleted and the warrior dies), otherwise the method returns true (because there is still remaining health and the warrior survives).
- The class does not override the **attack** method inherited from the parent class.

Finally, write test code that creates an array of **GameCharacter** objects. Of course, you should not store **GameCharacter** objects themselves, but only instances of the derived classes. Prompt the user for the details required to create two **GameCharacter** objects and add them to the array. Once all the game characters have been created and added, have the characters battle each other until one of them dies.

The battle starts by calling **attack** on the first game character in the array, yielding an integer value. Next the integer value is used as the parameter of the **takeHit** method that must be called on the second game character in the array.

Assuming the second game character survives, the roles are reversed. The **attack** method is called on the second game character in the array, yielding an integer value, and this value is used as the parameter of the **takeHit** method called on the first game character in the array.

The battle continues until one character dies. The details of each attack and the result of the hit (in terms of how many health points the attacked character has left) should be printed out at each step. The program should then print out the result of the battle (for each character should be described as dying or winning).

Be sure to utilise as much functionality from parent classes as possible, and do not re-implement functionality that exists in parent classes. Here is some example input and output for the specified program:

```
Enter a wizard (1) or a warrior (2): 1
Enter health: 15
Enter spell damage: 50
```

```
Enter a wizard (1) or a warrior (2): 2
Enter health: 60
Enter shield strength: 30
```

```
The battle begins!
```

```
Character 1 attacks for 50 damage
Character 2 has 40 health points left
```

```
Character 2 attacks for 1 damage
Character 1 has 0 health points left
```

```
Character 1 died
Character 2 wins
```

Ruby supplies shorthand for getters and setters, called accessor methods. Investigate this shorthand and use it to implement the get functionality for the **health** instance variable in the **GameCharacter** class. Additionally, you will also have to investigate arrays in Ruby.

6 Marking

The implementation will count a total of 5 marks. Submit the Ruby implementation to the appropriate assignment upload slot. Do not upload any additional files other than your source code. Both the implementation and the correct execution of the program will be taken into account. Your program code will be assessed during the practical sessions in the weeks of 18 May 2026 and 25 May 2026. Note that there is only one practical session in the week of 25 May 2026 (on the Monday).

References

- [1] Huw Collingbourne. *The Book of Ruby*. Self published, 2009.